



Problem Complexity Classification

Dr. Obed Noé Sámano Abonce



Problem classification

Computational Complexity

Big-O describes the efficiency of an algorithm, but complexity classes describe properties of computational problems.

Polynomial time provides the basis for defining classes such as P and NP.

Polynomial Time

- $O(n)$
- $O(n^2)$
- $O(n^3)$
- Generally scalable

Non-Polynomial Time

- $O(2^n)$
- $O(3^n)$
- $O(n!)$
- Quickly becomes impractical

Problem classification

Key idea

Solving a problem is not always as easy as verifying a solution

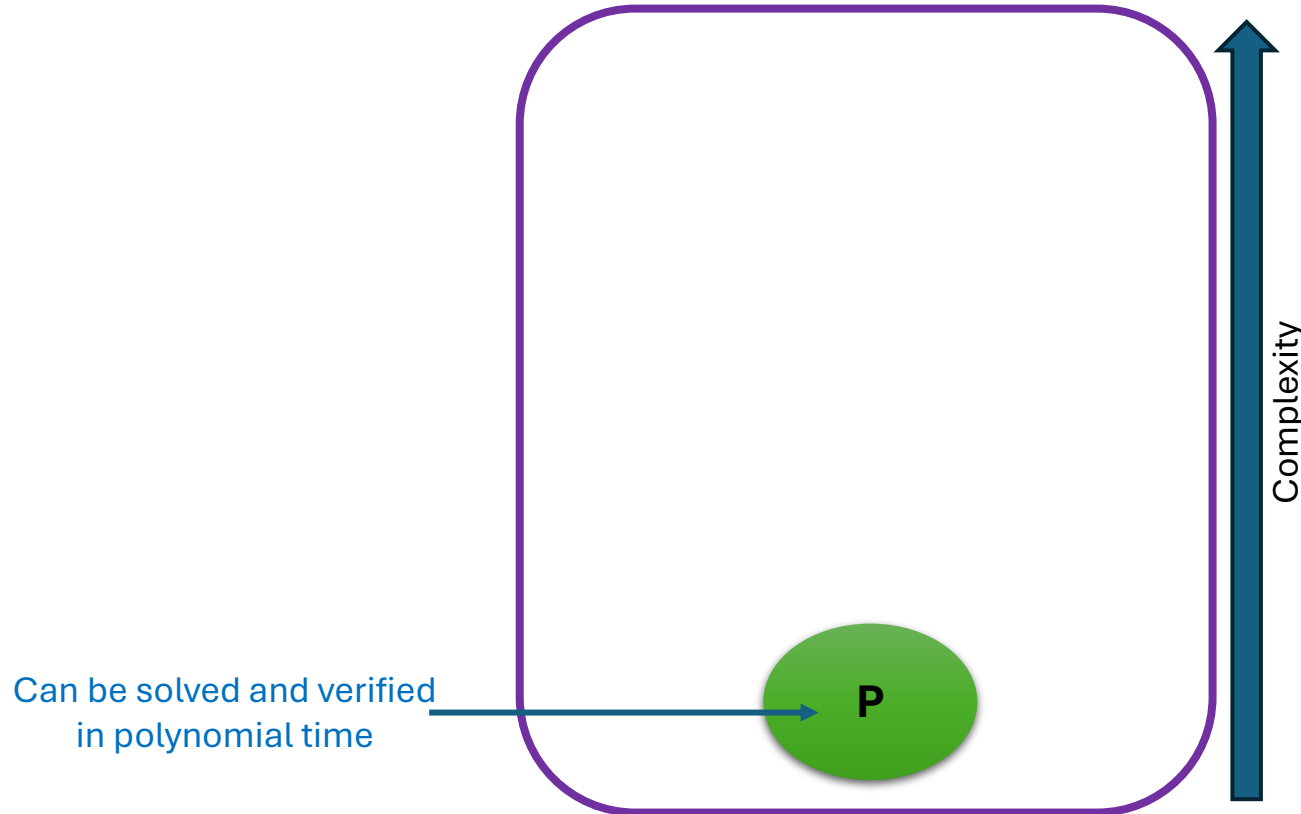
Some problems are hard to solve, but easy to verify.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Problem classification

P Problems (Polynomial Problems)

Computational Problems



Example:

Given a list of numbers and a target value, determine whether the target appears in the list.

[4, 7, 12, 18, 25, 31]

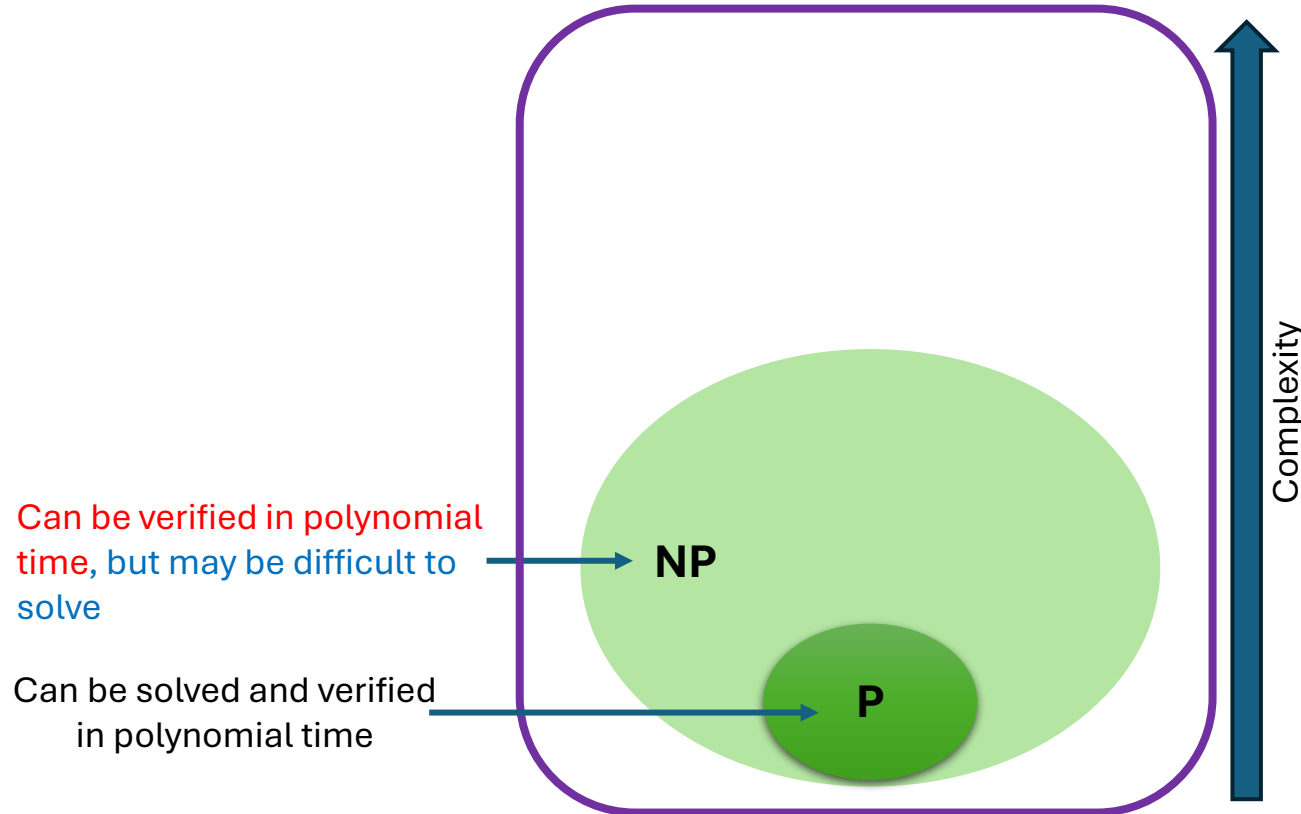
We can check each element. In the worst case we perform n comparisons

$$T(n) = O(n)$$

Problem classification

NP Problems (Non-Deterministic Polynomial Problems)

Computational Problems



Example: Travelling Salesman Person – Decision version

Given a set of cities and the distances between them, is there a route that visits every city exactly once, returns to the starting city, and has total length at most K ?

Proposed route $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$

Is the total distance ≤ 500 km?

Finding such route may be difficult, but verifying the proposed route can be done in polynomial time.

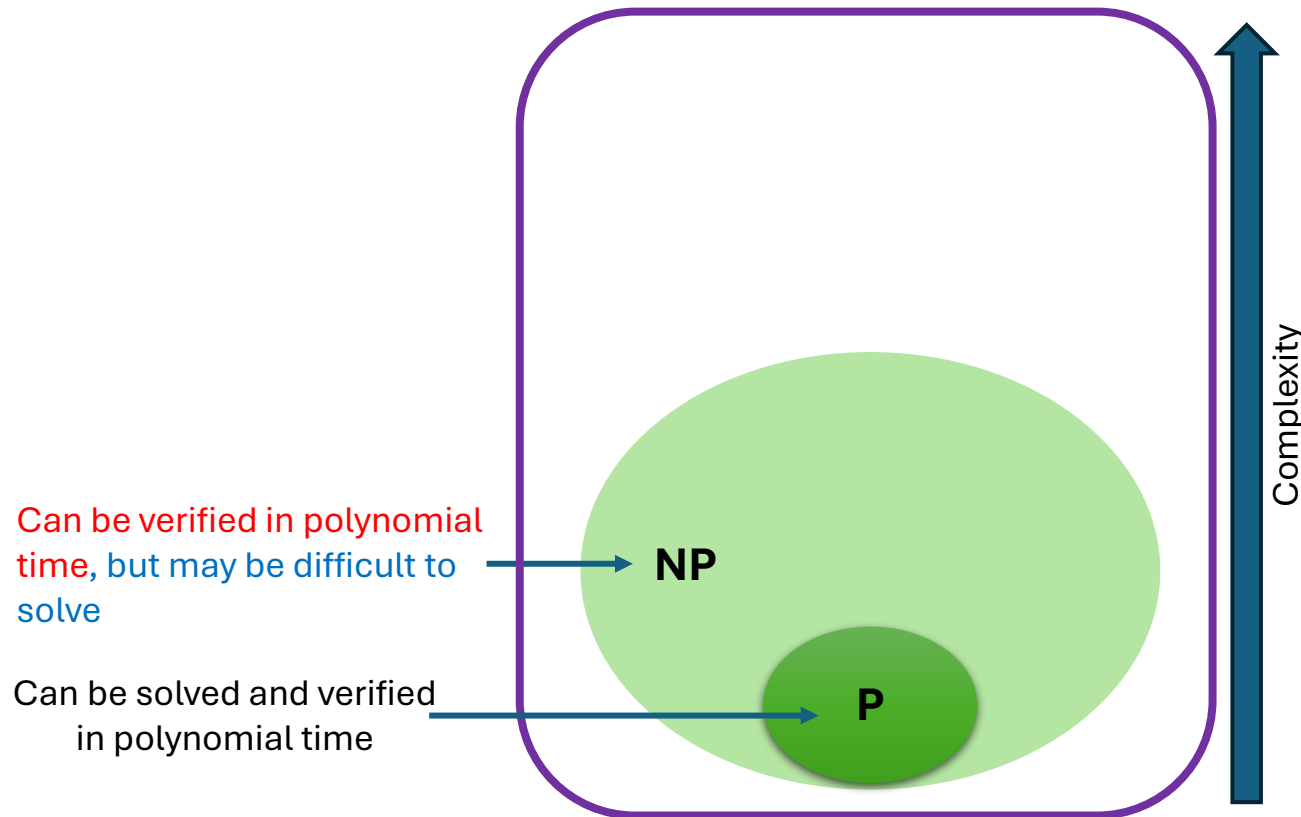
Solving: no known polynomial-time algorithm

Verifying a proposed solution: polynomial time

Problem classification

The Big Question: Is $P = NP$?

Computational Problems



If a solution can be verified in polynomial time, can it also be found in polynomial time?

$$P \subseteq NP$$

But is $P = NP$?

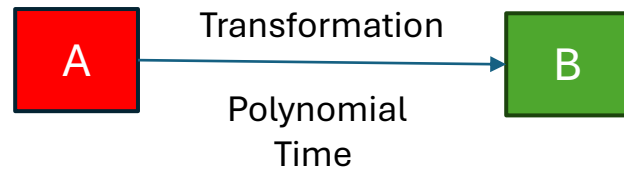
To understand why this question matters, we now need to look at NP-hard and NP-complete problems.

Problem classification

Polynomial-Time Reduction

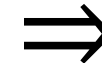
Problem **A** can be reduced to Problem **B** if any instance of **A** can be transformed into an instance of **B** in polynomial time.

$$A \leq_p B$$



If the transformation exists, then:

If we can
solve B
efficiently.



We can also
solve A
efficiently.

B is at least as hard as A

This allows us to compare the computational difficulty of problems.

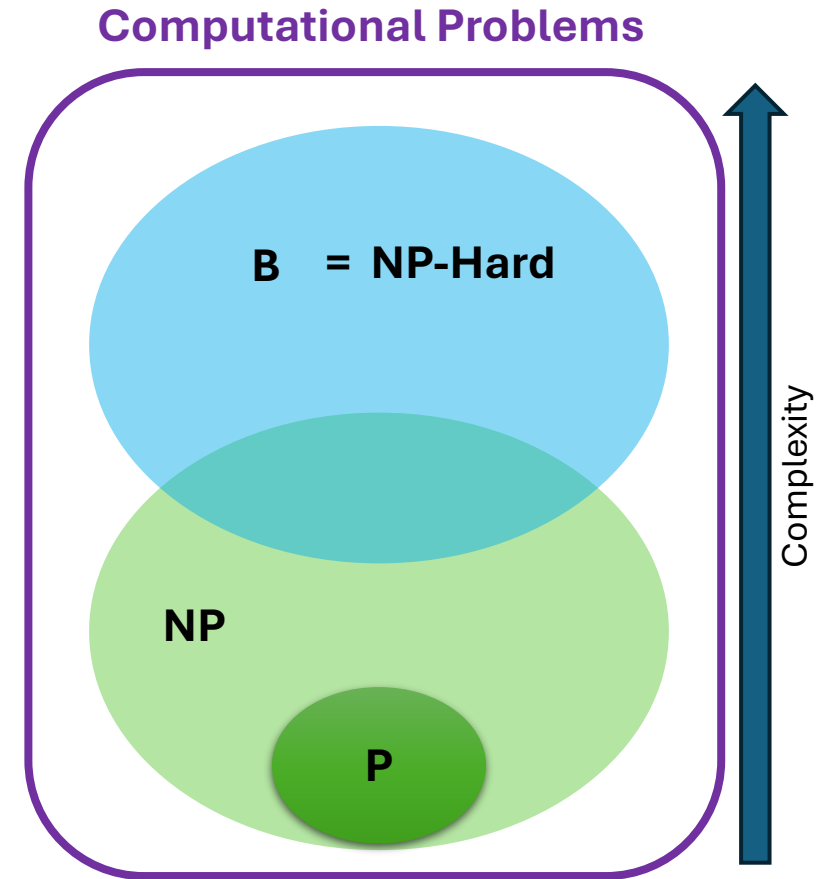
Problem classification

NP-Hard Problems

Now suppose that every problem in NP can be reduced to the same problem B.

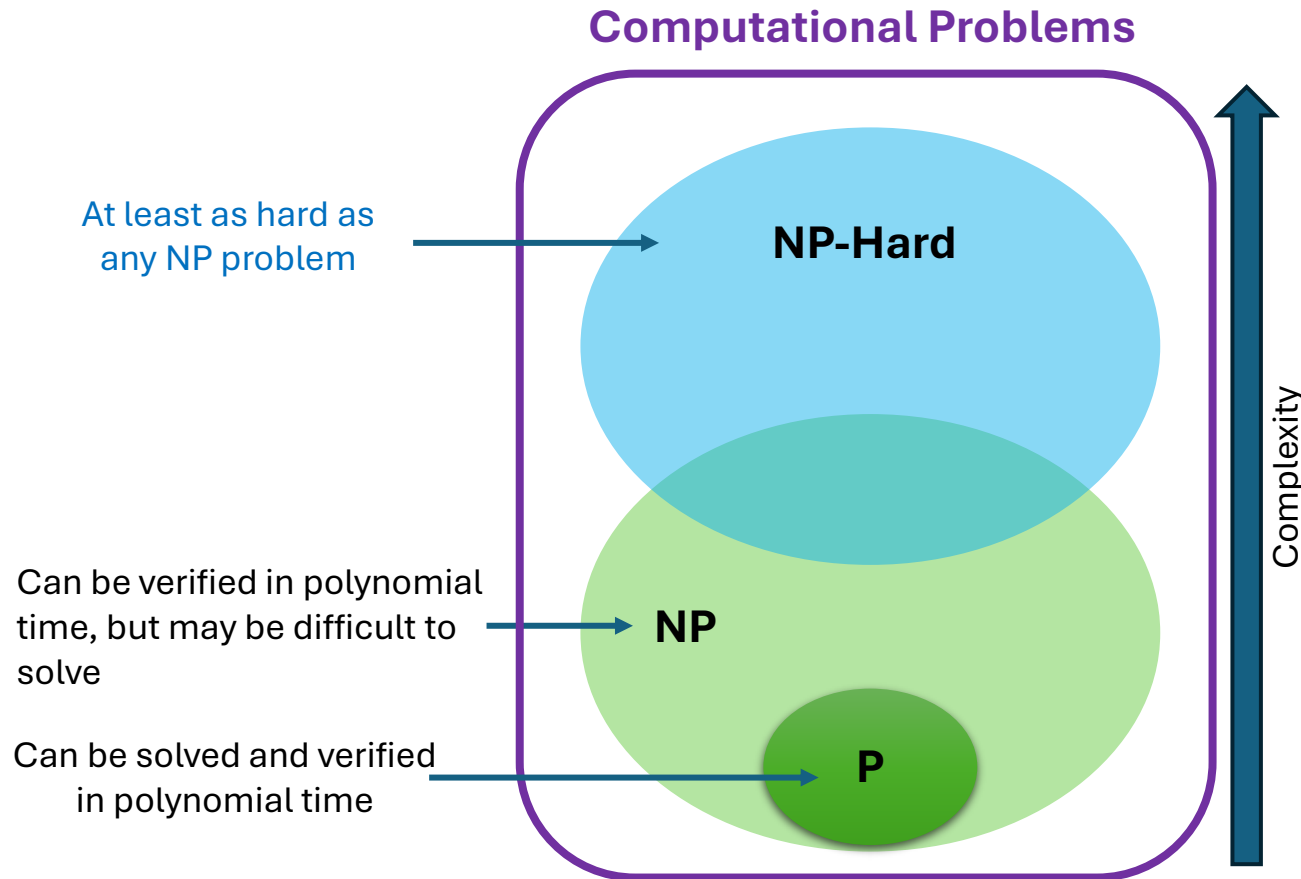
$$\forall A \in NP, \quad A \leq_p B$$

How hard would B be?



Problem classification

NP-Hard Problems



Example: Travelling Salesman – Optimization version

Find the shortest possible route that visits every city and returns to the starting point.

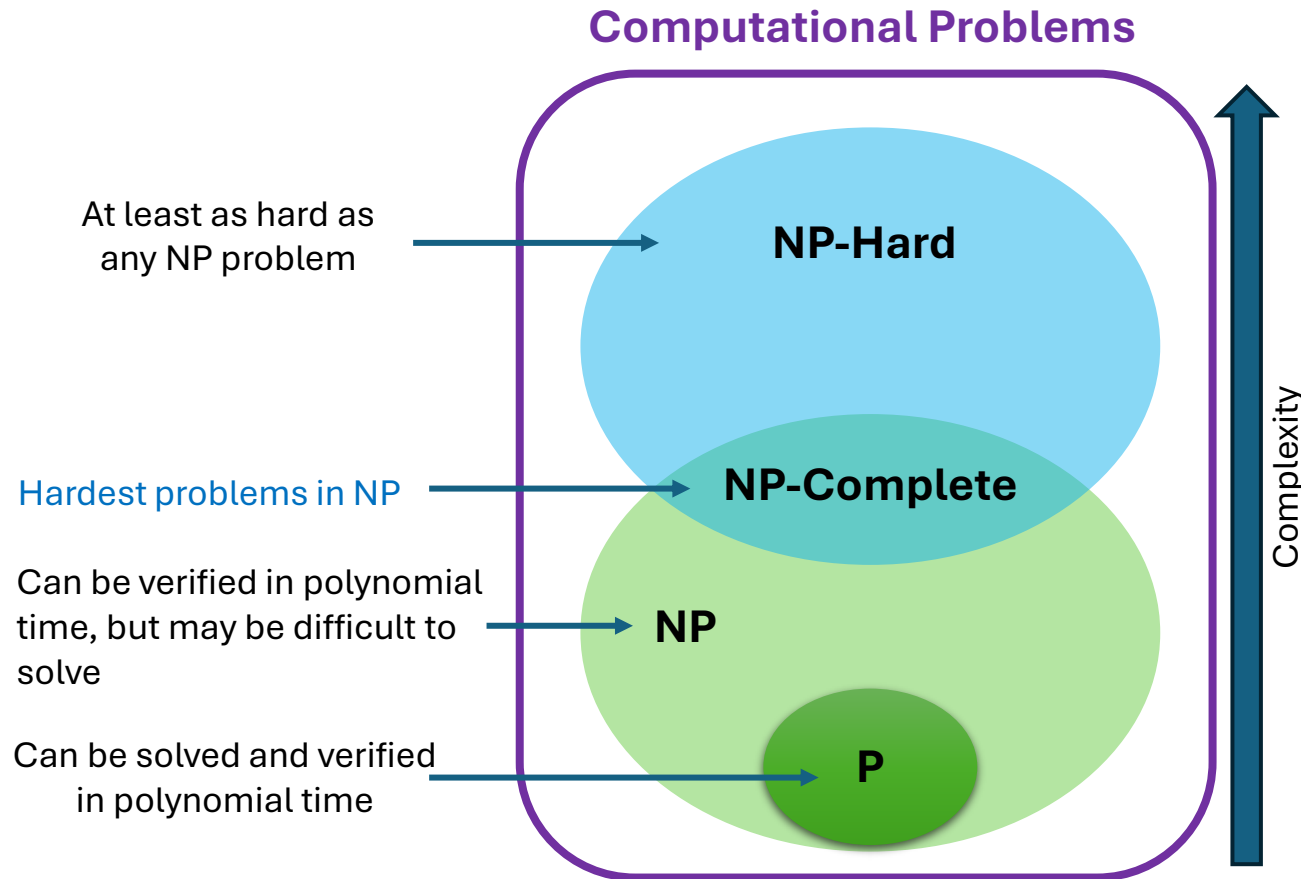
Solving: no known polynomial-time exact algorithm

Verifying a proposed route: polynomial time

Verifying that it is optimal: not known to be polynomial time

Problem classification

NP-Complete Problems



Example: Travelling Salesman – Decision version

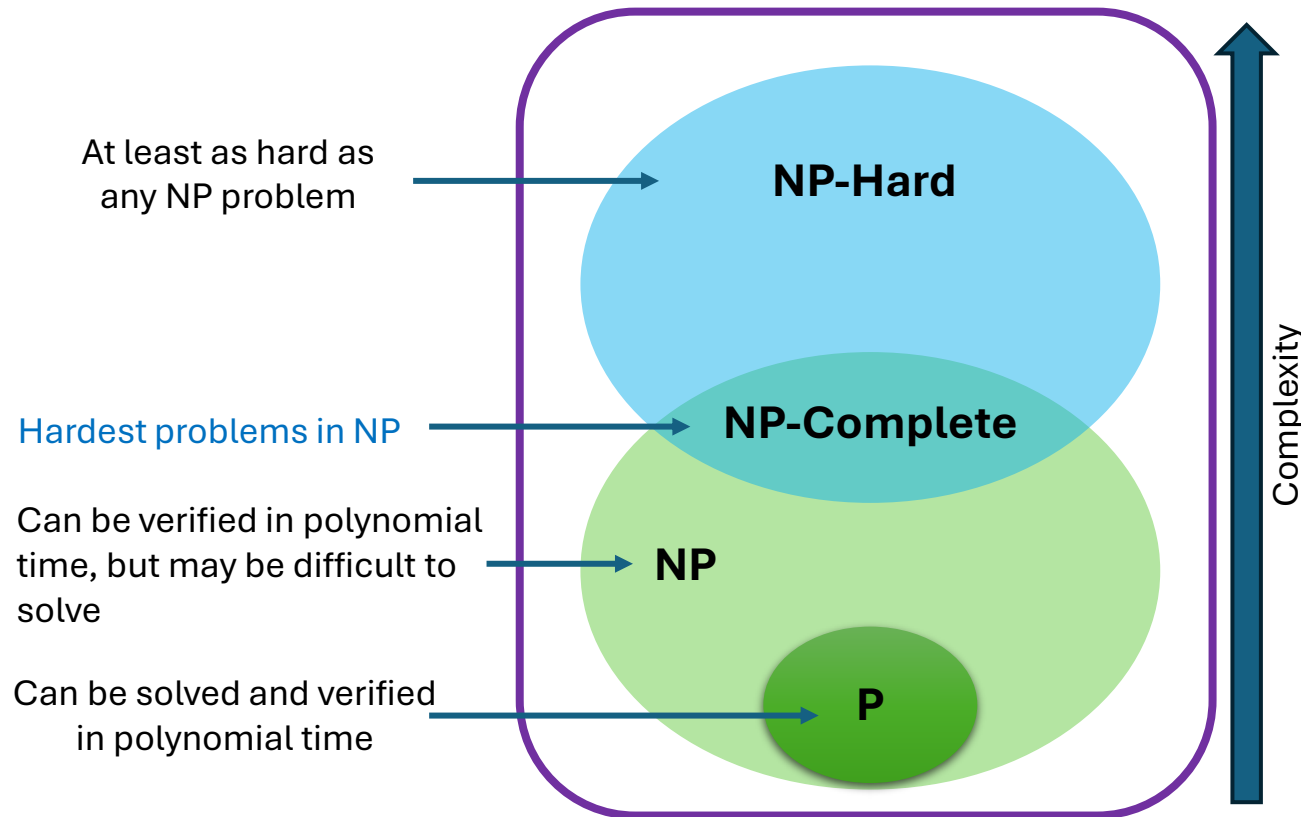
Is there a tour that visits every city and has total cost at most K ?

Solving: no known polynomial-time algorithm
Verifying a proposed route: polynomial time

Problem classification

Why Does NP-Complete Matter?

Computational Problems



If any NP-Complete problem can be solved in polynomial time, then every problem in NP can be solved in polynomial time:

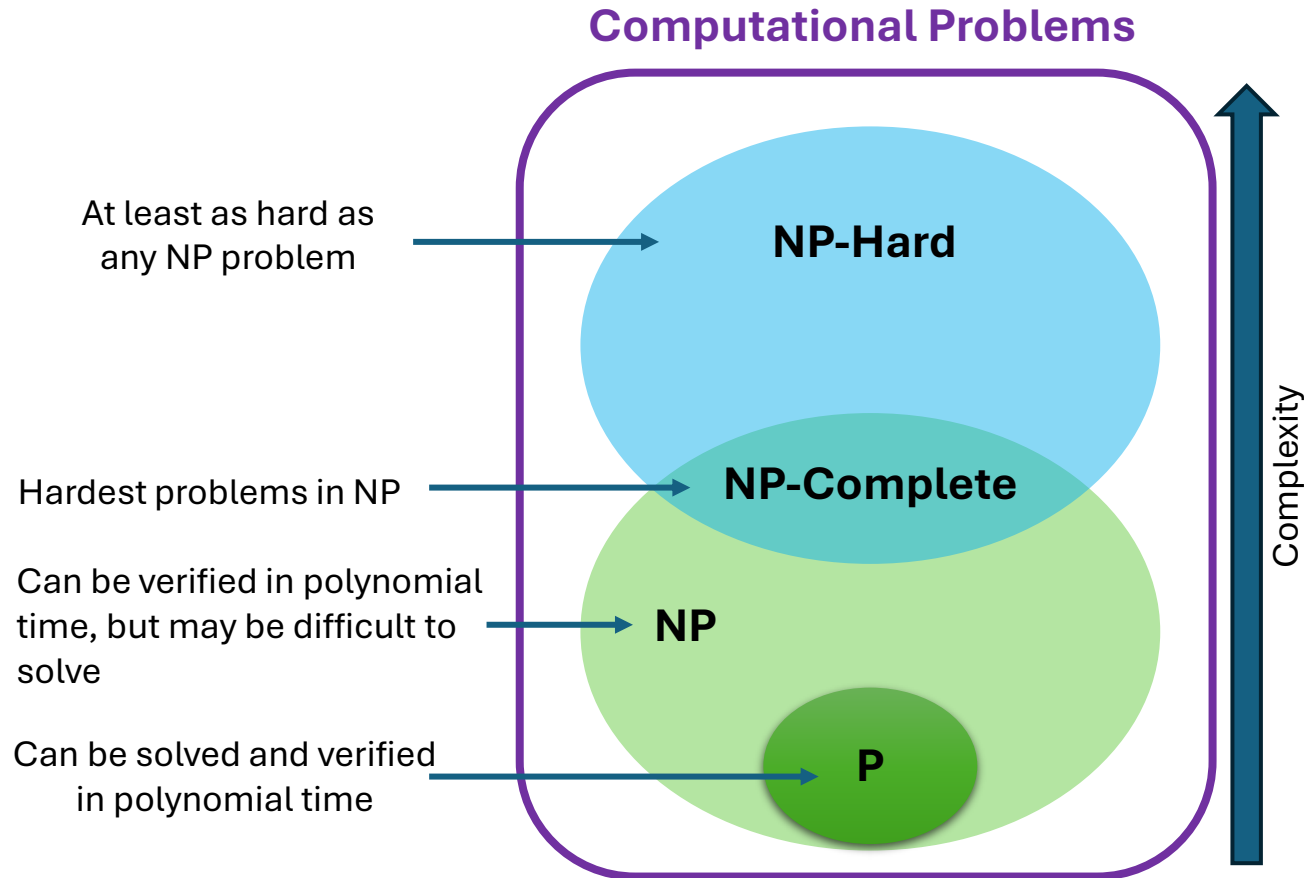
$$P = NP$$

If any NP-Complete problem is proven not to be in P:

$$P \neq NP$$

Problem classification

Putting It All Together



$$P = NP ?$$

One of the most important open problems in computer science.

References

S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021. Appendix A.1.2.